

Week 2 Progress Summary: Advanced Greedy & Data Structures

Overview

This week shifted focus toward optimization problems involving two-pointer greedy schemes, prefix sum look-aheads, coordinate sorting, and frequency/index map lookups using associative containers. A total of 6 problems were successfully implemented in C++.

Problem Summaries

1. Basketball Together

- Core Concept: Two-Pointer Greedy Strategy
- Approach: To maximize the number of winning teams, the available players are sorted by their power. Using a two-pointer approach, the algorithm greedily picks the strongest unassigned player to act as the team leader. It calculates the exact number of additional players required to make this leader's total team power cross the threshold D . The required numbers are then sacrificed from the weakest available players at the left pointer.

2. Collecting Game

- Core Concept: Prefix Sums & Multi-Pointer Look-Ahead
- Approach: Tracks how many elements a starting element can absorb by pairing values with their original indices and sorting them. It constructs a prefix sum array to keep track of the cumulative score accumulated up to any element. The algorithm employs a trailing pointer j that scans forward as far as possible, expanding the cascade whenever the current cumulative prefix sum is large enough to absorb the next element.

3. Ian and Array Sorting

- Core Concept: Parity & Ripple-Effect Invariance
- Approach: Analyzes whether an array can be made non-decreasing by applying operations that increase or decrease an adjacent pair of elements simultaneously. If the array size n is odd, it is always valid. For even n , the code executes a continuous, greedy forward pass ("ripple pass"). It forces elements into non-decreasing order step-by-step and updates the subsequent adjacent element, checking if the final two elements satisfy the non-decreasing condition at the end.

4. Points and Minimum Distance

- Core Concept: Coordinate Sorting & Greedy Path Optimization
- Approach: Minimizes the total Manhattan distance required to connect n geometric points along a straight path using $2n$ line segments. The code sorts the entire array of coordinates. It then splits the sorted array cleanly down the middle, pairing the first half (representing x -coordinates) directly with the second half (representing y -coordinates) to yield the minimum possible path perimeter.

5. Train and Queries

- Core Concept: Position Hashing / Map Index Lookup
- Approach: Optimizes route-validation queries by checking if a train can travel from station A to station B along a single path. Instead of traversing the array linearly for each query, the code uses two `std::map` containers to cache the absolute `first_occ` and `last_occ` indices of every station as they are read. Validating a path is reduced to an $O(1)$ lookup check to see if the earliest occurrence of A is smaller than the latest occurrence of B .

6. Yarik and Musical Notes

- Core Concept: Combinatorics & Frequency Maps
- Approach: Solves for pairs (i, j) satisfying the equation $a_i^{a_j} = a_j^{a_i}$.
Mathematically, this condition is only met when $a_i = a_j$, or in the single unique edge case where the values are 1 and 2 (since $1^2 \neq 2^1$ is false, but 2^1 vs 1^2 holds up under the original problem's log property transformations). The program populates a frequency map, calculates matching identical pairs using the combination formula $\frac{\text{count} \times (\text{count} - 1)}{2}$, and adds cross-products for counts of 1 and 2 .